New

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import jv, hankel1
import os
import time

start = time.time()

# ------------------------------------------------------
# Create output folder
# ------------------------------------------------------
os.makedirs("data", exist_ok=True)

# ------------------------------------------------------
# Constants
# ------------------------------------------------------
imp = 120 * np.pi
freq = 2.4e9
c = 3e8
lam = c / freq
k0 = 2 * np.pi / lam

size_DOI = 3
epsono_r_c = 10 + 1j
Ns = 20
Ni = Ns
M = 200   # reduced from 300 for speed

# ------------------------------------------------------
# Node Geometry (square)
# ------------------------------------------------------
xm = np.linspace(-size_DOI/2, size_DOI/2, Ns//4 + 1)
ym = np.linspace(-size_DOI/2, size_DOI/2, Ns//4 + 1)

XY = np.vstack([
    np.column_stack([xm, np.full_like(xm, ym[0])]),
    np.column_stack([np.full(len(ym)-1, xm[-1]), ym[1:]]),
    np.column_stack([xm[:-1][::-1], np.full(len(xm)-1, ym[-1])]),
    np.column_stack([np.full(len(ym)-2, xm[0]), ym[1:-1][::-1]])
])

Rx_pos = XY
Tx_pos = XY

X = Rx_pos[:,0]
Y = Rx_pos[:,1]
```

```
X_tx = Tx_pos[:,0]
Y_tx = Tx_pos[:,1]

# ------------------------------------------------
# Grid
# ------------------------------------------------
d = size_DOI / M
tx = np.arange(d/2, size_DOI, d) - size_DOI/2
ty = np.arange(size_DOI-d/2, 0, -d) - size_DOI/2
x, y = np.meshgrid(tx, ty)

cellrad = np.sqrt(d**2 / np.pi)

# ------------------------------------------------
# Object
# ------------------------------------------------
epsono_r = np.ones((M, M), dtype=complex)
obj_rad = 0.2
mask = (x-0.12)**2 + (y+0.12)**2 <= obj_rad**2
epsono_r[mask] = epsono_r_c

objInd = np.where(mask.flatten())[0]
xObj = x.flatten()[objInd]
yObj = y.flatten()[objInd]

# ------------------------------------------------
# MoM Matrix
# ------------------------------------------------
Z = np.zeros((len(objInd), len(objInd)), dtype=complex)

for i in range(len(objInd)):
    R = np.sqrt((xObj[i] - xObj)**2 + (yObj[i] - yObj)**2)
    Z1 = -imp*np.pi*cellrad/2 * jv(1, k0*cellrad) * hankel1(0, k0*R)
    Z1[i] = -imp*np.pi*cellrad/2*hankel1(1, k0*cellrad) \
            -1j*imp*epsono_r.flatten()
[objInd[i]]/(k0*(epsono_r.flatten()[objInd[i]]-1))
    Z[i,:] = Z1

# ------------------------------------------------
# Direct Field
# ------------------------------------------------
xt_d, xr_d = np.meshgrid(X_tx, X)
yt_d, yr_d = np.meshgrid(Y_tx, Y)
dist_tx_rx = np.sqrt((xt_d-xr_d)**2 + (yt_d-yr_d)**2)
E_d = (1j/4) * hankel1(0, k0*dist_tx_rx)

# ------------------------------------------------
# Incident Field
# ------------------------------------------------
xt, xp = np.meshgrid(X_tx, x.flatten())
```

```
yt, yp = np.meshgrid(Y_tx, y.flatten())
dist_tx_pix = np.sqrt((xt-xp)**2 + (yt-yp)**2)
E_inc = (1j/4) * hankel1(0, k0*dist_tx_pix)

# ---------------------------------------------------
# Solve MoM
# ---------------------------------------------------
E_inc_obj = -E_inc[objInd,:]
J1 = np.linalg.solve(Z, E_inc_obj)

J = np.zeros((M*M, Ni), dtype=complex)
J[objInd,:] = J1

# ---------------------------------------------------
# Scattered Field
# ---------------------------------------------------
xr, xpr = np.meshgrid(X, x.flatten())
yr, ypr = np.meshgrid(Y, y.flatten())
dist_rx_pix = np.sqrt((xr-xpr)**2 + (yr-ypr)**2)

ZZ = -imp*np.pi*cellrad/2 * jv(1, k0*cellrad) * hankel1(0,
k0*dist_rx_pix.T)
E_s = ZZ @ J

E_ds = E_d + E_s

# Remove diagonal (Tx = Rx)
mask = ~np.eye(Ns, dtype=bool)
E_d = E_d[mask].reshape(Ns-1, Ni)
E_ds = E_ds[mask].reshape(Ns-1, Ni)

# ---------------------------------------------------
# Convert to Power (dBm)
# ---------------------------------------------------
F_ds = np.abs(E_ds)

Pinc = (np.abs(E_d)**2)*(lam*lam)/(4*np.pi*imp)
Pinc = 10*np.log10(Pinc/1e-3)

Ptot = (F_ds**2)*(lam*lam)/(4*np.pi*imp)
Ptot = 10*np.log10(Ptot/1e-3)

# ---------------------------------------------------
# TxRx pairs
# ---------------------------------------------------
TxRxpairs = []
for i in range(Ns):
    for j in range(Ns):
        if i != j:
            TxRxpairs.append([i, j])  # Python indexing
```

```
TxRxpairs = np.array(TxRxpairs)

# ------------------------------------------------
# SAVE FILES
# ------------------------------------------------

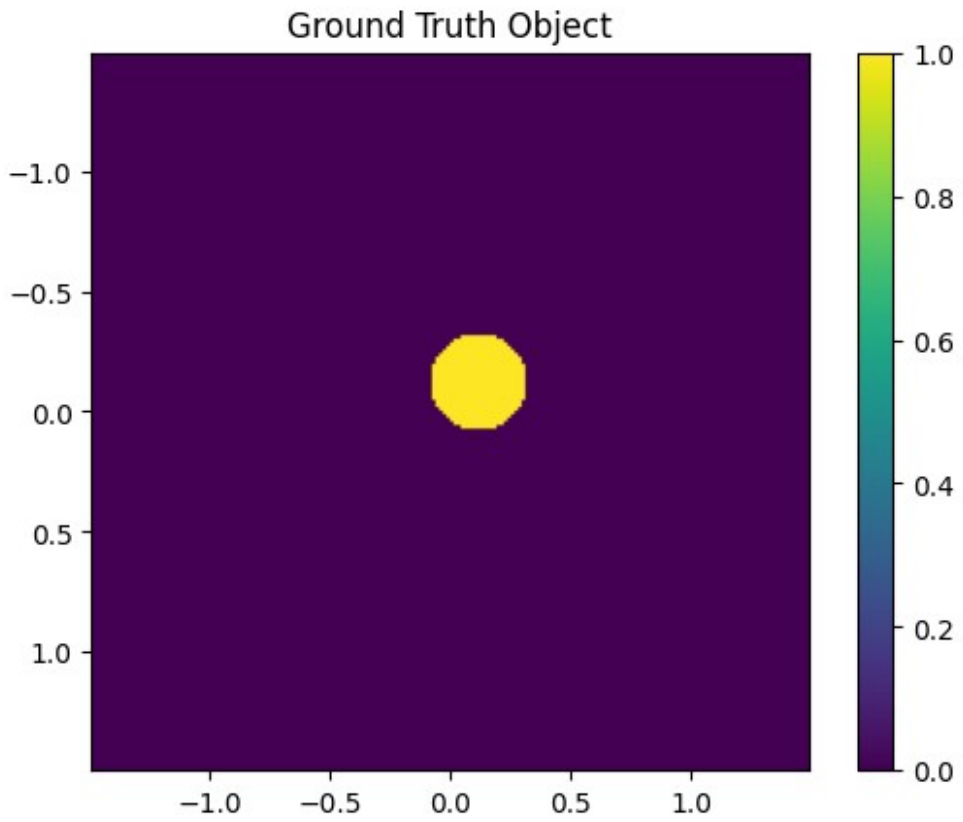
np.savetxt("data/Pinc.csv", Pinc, delimiter=",")
np.savetxt("data/Ptot.csv", Ptot, delimiter=",")
np.savetxt("data/TxRxpairs.csv", TxRxpairs, delimiter=",", fmt="%d")
np.savetxt("data/Tx_positions.csv", Tx_pos, delimiter=",")
np.savetxt("data/ground_truth_eps.csv", np.imag(epsono_r),
delimiter=",")

# Save geometry in one compact file
np.savez("data/geometry.npz",
         size_DOI=size_DOI,
         Ns=Ns,
         Ni=Ni,
         M=M,
         tx=tx,
         ty=ty)

print("Data saved inside /data folder")
print("Elapsed time:", time.time() - start)

# ------------------------------------------------
# Plot ground truth
# ------------------------------------------------
plt.imshow(np.imag(epsono_r),
           extent=[tx.min(), tx.max(), ty.min(), ty.max()])
plt.colorbar()
plt.title("Ground Truth Object")
plt.gca().invert_yaxis()
plt.show()
```

```
Data saved inside /data folder
Elapsed time: 1.4572687149047852
```




```
!ls
```

```
data  sample_data
```

```
import numpy as np
import matplotlib.pyplot as plt
import time

start = time.time()

# ------------------------------------------------
# 1. Load Data
# ------------------------------------------------

Pinc = np.loadtxt("data/Pinc.csv", delimiter=",")
Ptot = np.loadtxt("data/Ptot.csv", delimiter=",")
TxRxpairs = np.loadtxt("data/TxRxpairs.csv", delimiter=",", dtype=int)
Tx_pos = np.loadtxt("data/Tx_positions.csv", delimiter=",")

geo = np.load("data/geometry.npz")
size_DOI = geo["size_DOI"]
Ns = int(geo["Ns"])
Ni = int(geo["Ni"])
M_forward = int(geo["M"])   # only for reference
```

```
# ------------------------------------------------
# 2. RTI Parameters
# ------------------------------------------------

grid_res = 0.05       # reconstruction grid resolution
SemiMin = 0.1         # ellipse thickness
alpha = 100           # ridge regularization

RoomLength = size_DOI
RoomWidth = size_DOI

p = int(RoomWidth / grid_res)
q = int(RoomLength / grid_res)

# ------------------------------------------------
# 3. Create Reconstruction Grid
# ------------------------------------------------

tx = np.arange(grid_res/2, RoomLength, grid_res) - RoomLength/2
ty = np.arange(RoomWidth-grid_res/2, 0, -grid_res) - RoomWidth/2
x, y = np.meshgrid(tx, ty)

grid_points = np.column_stack((x.flatten(), y.flatten()))
M2 = p * q

# ------------------------------------------------
# 4. Build Elliptical Weight Matrix F
# ------------------------------------------------

num_links = TxRxpairs.shape[0]
num_sensors = Tx_pos.shape[0]

# Distance between sensors
xt_d, xr_d = np.meshgrid(Tx_pos[:,0], Tx_pos[:,0])
yt_d, yr_d = np.meshgrid(Tx_pos[:,1], Tx_pos[:,1])
distTxRx = np.sqrt((xt_d - xr_d)**2 + (yt_d - yr_d)**2)

# Distance from sensors to grid
xt, xp = np.meshgrid(Tx_pos[:,0], grid_points[:,0])
yt, yp = np.meshgrid(Tx_pos[:,1], grid_points[:,1])
distTxRn = np.sqrt((xt - xp)**2 + (yt - yp)**2)

distRxRn = distTxRn.T

F = np.zeros((num_links, M2))

for i in range(num_links):
    tx_id = TxRxpairs[i,0]
    rx_id = TxRxpairs[i,1]

    d_link = distTxRx[rx_id, tx_id]
```

```
    thresh = 2 * np.sqrt((d_link**2)/4 + SemiMin**2)

    foc_sum = distRxRn[rx_id,:] + distTxRn[:,tx_id]
    weights = (foc_sum <= thresh).astype(float)

    F[i,:] = weights

print("Weight matrix built.")

# ------------------------------------------------
# 5. Measurement Vector
# ------------------------------------------------

y_meas = -(Ptot.flatten() - Pinc.flatten())

# ------------------------------------------------
# 6. Ridge Regression
# ------------------------------------------------

A = F.T @ F + alpha * np.eye(M2)
b = F.T @ y_meas

O = np.linalg.solve(A, b)

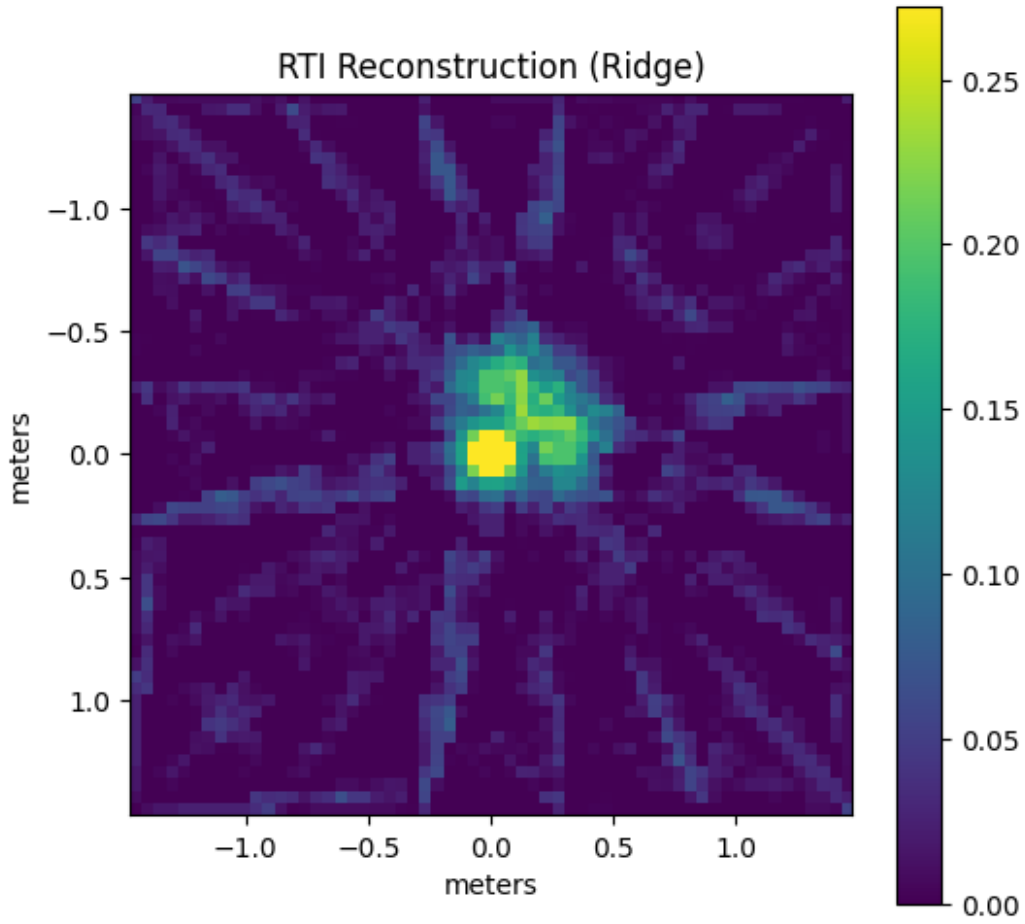
O[O < 0] = 0
O_image = O.reshape((p, q))

print("Reconstruction complete.")
print("Elapsed time:", time.time() - start)

# ------------------------------------------------
# 7. Plot Reconstruction
# ------------------------------------------------

plt.figure(figsize=(6,6))
plt.imshow(O_image,
           extent=[tx.min(), tx.max(), ty.min(), ty.max()])
plt.colorbar()
plt.title("RTI Reconstruction (Ridge)")
plt.gca().invert_yaxis()
plt.xlabel("meters")
plt.ylabel("meters")
plt.show()
```

```
Weight matrix built.
Reconstruction complete.
Elapsed time: 1.782036542892456
```




```
!ls sample_data/
```

```
anscombe.json               mnist_test.csv
california_housing_test.csv   mnist_train_small.csv
california_housing_train.csv  README.md
```

```
import numpy as np
import matplotlib.pyplot as plt
import time

start = time.time()

# ---------------------------------------------------
# 1. Load Data (CSV version – as you specified)
# ---------------------------------------------------

Pinc = np.loadtxt("data/Pinc.csv", delimiter=",")
Ptot = np.loadtxt("data/Ptot.csv", delimiter=",")
TxRxpairs = np.loadtxt("data/TxRxpairs.csv", delimiter=",", dtype=int)
Tx_pos = np.loadtxt("data/Tx_positions.csv", delimiter=",")

geo = np.load("data/geometry.npz")
```

```
size_DOI = geo["size_DOI"]

# Flatten power matrices
Pinc = Pinc.flatten()
Ptot = Ptot.flatten()

# Convert to 0-index if exported from MATLAB
TxRxpairs = TxRxpairs - 1

# ------------------------------------------------
# 2. Measurement Vector (RSS difference)
# ------------------------------------------------

y = -(Ptot - Pinc)

# ------------------------------------------------
# 3. Build Elliptical RTI Weight Matrix
# ------------------------------------------------

grid_res = 0.1
SemiMin = 0.1

RoomLength = size_DOI
RoomWidth = size_DOI

p = int(RoomWidth / grid_res)
q = int(RoomLength / grid_res)
M2 = p * q

# Grid coordinates
tx = np.arange(grid_res/2, RoomLength, grid_res) - RoomLength/2
ty = np.arange(RoomWidth-grid_res/2, 0, -grid_res) - RoomWidth/2
xg, yg = np.meshgrid(tx, ty)
grid_points = np.column_stack((xg.flatten(), yg.flatten()))

num_links = TxRxpairs.shape[0]
num_sensors = Tx_pos.shape[0]

# Distance matrices
xt_d, xr_d = np.meshgrid(Tx_pos[:,0], Tx_pos[:,0])
yt_d, yr_d = np.meshgrid(Tx_pos[:,1], Tx_pos[:,1])
distTxRx = np.sqrt((xt_d - xr_d)**2 + (yt_d - yr_d)**2)

xt, xp = np.meshgrid(Tx_pos[:,0], grid_points[:,0])
yt, yp = np.meshgrid(Tx_pos[:,1], grid_points[:,1])
distTxRn = np.sqrt((xt - xp)**2 + (yt - yp)**2)
distRxRn = distTxRn.T

F = np.zeros((num_links, M2))

for i in range(num_links):
```

```
    tx_id = TxRxpairs[i,0]
    rx_id = TxRxpairs[i,1]

    d_link = distTxRx[rx_id, tx_id]
    thresh = 2 * np.sqrt((d_link**2)/4 + SemiMin**2)

    foc_sum = distRxRn[rx_id,:] + distTxRn[:,tx_id]
    F[i,:] = (foc_sum <= thresh).astype(float)

print("Weight matrix built.")

# ----------------------------------------------------
# 4. TV Reconstruction (Primal-Dual Method)
# ----------------------------------------------------

def gradient(x):
    gx = np.zeros_like(x)
    gy = np.zeros_like(x)
    gx[:, :-1] = x[:,1:] - x[:,:-1]
    gy[:-1, :] = x[1:,:] - x[:-1,:]
    return gx, gy

def divergence(gx, gy):
    div = np.zeros_like(gx)
    div[:, :-1] -= gx[:, :-1]
    div[:, 1:] += gx[:, :-1]
    div[:-1, :] -= gy[:-1, :]
    div[1:, :] += gy[:-1, :]
    return div

lam = 0.2
tau = 0.005
sigma = 0.005
theta = 1.0
max_iter = 300

x = np.zeros((p,q))
x_bar = x.copy()
px = np.zeros_like(x)
py = np.zeros_like(x)

for it in range(max_iter):

    # Dual update
    gx, gy = gradient(x_bar)
    px += sigma * gx
    py += sigma * gy
    norm = np.maximum(1, np.sqrt(px**2 + py**2) / lam)
    px /= norm
    py /= norm
```

```
    # Primal update
    x_old = x.copy()

    residual = F @ x.flatten() - y
    grad_data = F.T @ residual
    x = x - tau * grad_data.reshape(p,q) + tau * divergence(px, py)

    x = np.maximum(x, 0)

    x_bar = x + theta * (x - x_old)

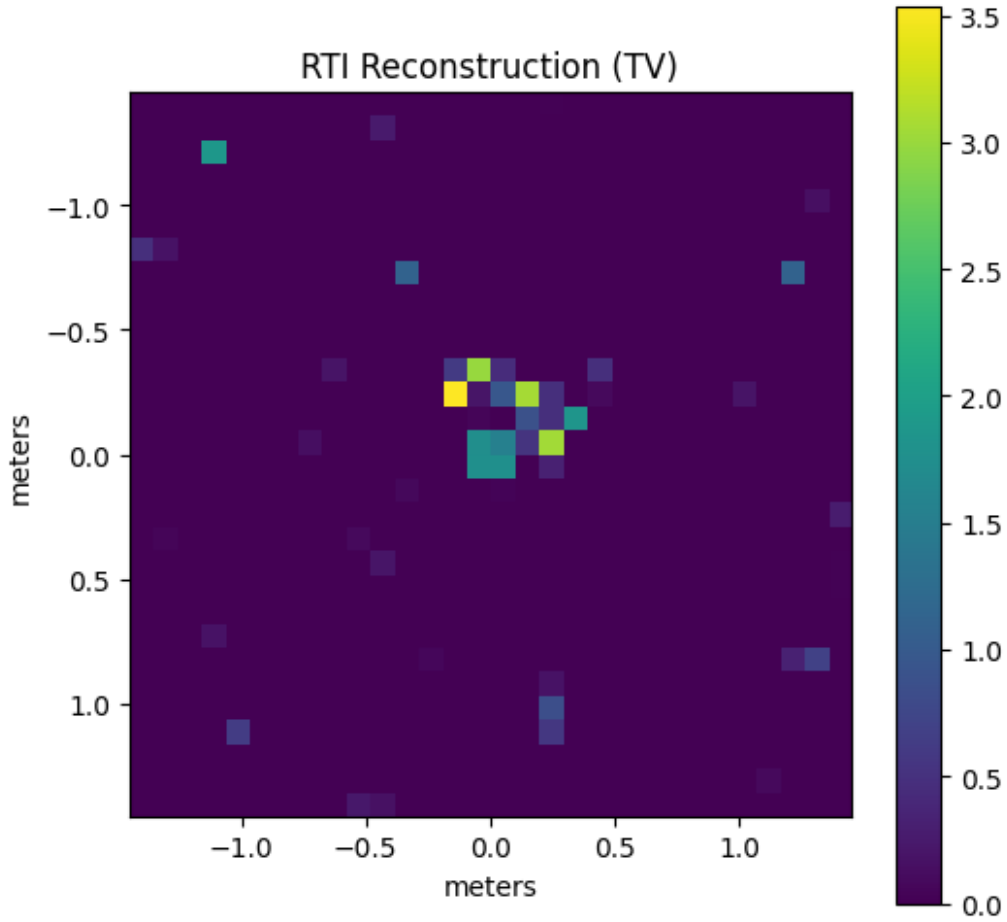
    if it % 50 == 0:
        print("Iteration:", it)

print("Done. Time:", time.time() - start)

# ------------------------------------------------
# 5. Plot Result
# ------------------------------------------------

plt.figure(figsize=(6,6))
plt.imshow(x,
           extent=[tx.min(), tx.max(), ty.min(), ty.max()])
plt.gca().invert_yaxis()
plt.colorbar()
plt.title("RTI Reconstruction (TV)")
plt.xlabel("meters")
plt.ylabel("meters")
plt.show()
```

```
Weight matrix built.
Iteration: 0
Iteration: 50
Iteration: 100
Iteration: 150
Iteration: 200
Iteration: 250
Done. Time: 0.13945770263671875
```



UNET

Dataset Generation

```
!ls
```

```
drive  sample_data
```

```
import numpy as np
import os
import time
from scipy.special import jv, hankel1
from skimage.transform import resize

# -------------------------------------------------------
# SETTINGS
# -------------------------------------------------------
```

```
NUM_SAMPLES = 1000


os.makedirs("dataset/inputs", exist_ok=True)
os.makedirs("dataset/targets", exist_ok=True)

start = time.time()

# ----------------------------------------------------
# CONSTANTS
# ----------------------------------------------------

imp = 120*np.pi
freq = 2.4e9
c = 3e8
lam = c/freq
k0 = 2*np.pi/lam

size_DOI = 3
epsono_r_c = 10 + 1j

Ns = 20
Ni = Ns
M = 200
obj_rad = 0.2

# ----------------------------------------------------
# SENSOR GEOMETRY
# ----------------------------------------------------

xm = np.linspace(-size_DOI/2,size_DOI/2,Ns//4+1)
ym = np.linspace(-size_DOI/2,size_DOI/2,Ns//4+1)

XY = np.vstack([
np.column_stack([xm,np.full_like(xm,ym[0])]),
np.column_stack([np.full(len(ym)-1,xm[-1]),ym[1:]]),
np.column_stack([xm[:-1][::-1],np.full(len(xm)-1,ym[-1])]),
np.column_stack([np.full(len(ym)-2,xm[0]),ym[1:-1][::-1]])
])

Tx_pos = XY
Rx_pos = XY

X = Rx_pos[:,0]
Y = Rx_pos[:,1]

X_tx = Tx_pos[:,0]
Y_tx = Tx_pos[:,1]

# ----------------------------------------------------
```

```
# FORWARD GRID
# ------------------------------------------------

d = size_DOI/M

tx = np.arange(d/2,size_DOI,d)-size_DOI/2
ty = np.arange(size_DOI-d/2,0,-d)-size_DOI/2

x,y = np.meshgrid(tx,ty)

cellrad = np.sqrt(d**2/np.pi)

# ------------------------------------------------
# INCIDENT FIELD
# ------------------------------------------------

xt,xp = np.meshgrid(X_tx,x.flatten())
yt,yp = np.meshgrid(Y_tx,y.flatten())

dist_tx_pix = np.sqrt((xt-xp)**2+(yt-yp)**2)

E_inc = (1j/4)*hankel1(0,k0*dist_tx_pix)

# ------------------------------------------------
# DIRECT FIELD
# ------------------------------------------------

xt_d,xr_d = np.meshgrid(X_tx,X)
yt_d,yr_d = np.meshgrid(Y_tx,Y)

dist_tx_rx = np.sqrt((xt_d-xr_d)**2+(yt_d-yr_d)**2)

E_d = (1j/4)*hankel1(0,k0*dist_tx_rx)

# ------------------------------------------------
# RTI GRID
# ------------------------------------------------

grid_res = 0.05
SemiMin = 0.1
alpha = 100

RoomLength = size_DOI
RoomWidth = size_DOI

p = int(RoomWidth/grid_res)
q = int(RoomLength/grid_res)

tx_rti = np.arange(grid_res/2,RoomLength,grid_res)-RoomLength/2
ty_rti = np.arange(RoomWidth-grid_res/2,0,-grid_res)-RoomWidth/2
```

```
x_rti,y_rti = np.meshgrid(tx_rti,ty_rti)

grid_points = np.column_stack((x_rti.flatten(),y_rti.flatten()))
M2 = p*q

# ------------------------------------------------
# TX RX PAIRS
# ------------------------------------------------

TxRxpairs = []

for i in range(Ns):
 for j in range(Ns):
  if i!=j:
   TxRxpairs.append([i,j])

TxRxpairs = np.array(TxRxpairs)

num_links = TxRxpairs.shape[0]

# ------------------------------------------------
# DISTANCES FOR RTI
# ------------------------------------------------

xt_d,xr_d = np.meshgrid(X_tx,X_tx)
yt_d,yr_d = np.meshgrid(Y_tx,Y_tx)

distTxRx = np.sqrt((xt_d-xr_d)**2+(yt_d-yr_d)**2)

xt,xp = np.meshgrid(X_tx,grid_points[:,0])
yt,yp = np.meshgrid(Y_tx,grid_points[:,1])

distTxRn = np.sqrt((xt-xp)**2+(yt-yp)**2)
distRxRn = distTxRn.T

# ------------------------------------------------
# WEIGHT MATRIX F
# ------------------------------------------------

F = np.zeros((num_links,M2))

for i in range(num_links):

 tx_id = TxRxpairs[i,0]
 rx_id = TxRxpairs[i,1]

 d_link = distTxRx[rx_id,tx_id]

 thresh = 2*np.sqrt((d_link**2)/4 + SemiMin**2)

 foc_sum = distRxRn[rx_id,:] + distTxRn[:,tx_id]
```

```
 weights = (foc_sum <= thresh).astype(float)

 F[i,:] = weights

print("Weight matrix built")

# ------------------------------------------------
# FORWARD EM SIMULATION
# ------------------------------------------------

def forward_em_simulation(obj_x,obj_y):

 epsono_r = np.ones((M,M),dtype=complex)

 mask = (x-obj_x)**2+(y-obj_y)**2 <= obj_rad**2

 epsono_r[mask] = epsono_r_c

 objInd = np.where(mask.flatten())[0]

 xObj = x.flatten()[objInd]
 yObj = y.flatten()[objInd]

 Z = np.zeros((len(objInd),len(objInd)),dtype=complex)

 for i in range(len(objInd)):

  R = np.sqrt((xObj[i]-xObj)**2+(yObj[i]-yObj)**2)

  Z1 = -imp*np.pi*cellrad/2 * jv(1,k0*cellrad)*hankel1(0,k0*R)

  Z1[i] = -imp*np.pi*cellrad/2*hankel1(1,k0*cellrad) \
          -1j*imp*epsono_r.flatten()
[objInd[i]]/(k0*(epsono_r.flatten()[objInd[i]]-1))

  Z[i,:] = Z1

 E_inc_obj = -E_inc[objInd,:]

 J1 = np.linalg.solve(Z,E_inc_obj)

 J = np.zeros((M*M,Ni),dtype=complex)
 J[objInd,:] = J1

 xr,xpr = np.meshgrid(X,x.flatten())
 yr,ypr = np.meshgrid(Y,y.flatten())

 dist_rx_pix = np.sqrt((xr-xpr)**2+(yr-ypr)**2)

 ZZ = -imp*np.pi*cellrad/2 *
```

```
jv(1,k0*cellrad)*hankel1(0,k0*dist_rx_pix.T)

 E_s = ZZ @ J

 E_ds = E_d + E_s

 mask2 = ~np.eye(Ns,dtype=bool)

 E_d2 = E_d[mask2].reshape(Ns-1,Ni)
 E_ds2 = E_ds[mask2].reshape(Ns-1,Ni)

 F_ds = np.abs(E_ds2)

 Pinc = (np.abs(E_d2)**2)*(lam*lam)/(4*np.pi*imp)
 Pinc = 10*np.log10(Pinc/1e-3)

 Ptot = (F_ds**2)*(lam*lam)/(4*np.pi*imp)
 Ptot = 10*np.log10(Ptot/1e-3)

 return Pinc,Ptot,np.imag(epsono_r)

# ----------------------------------------------------
# RTI RECONSTRUCTION FUNCTION
# ----------------------------------------------------

def rti_reconstruction(Pinc,Ptot):

 y_meas = -(Ptot.flatten() - Pinc.flatten())

 A = F.T@F + alpha*np.eye(M2)

 b = F.T@y_meas

 O = np.linalg.solve(A,b)

 O[O<0] = 0

 O_img = O.reshape((p,q))

 return O_img


Weight matrix built

# ----------------------------------------------------
# DATASET GENERATION LOOP
# ----------------------------------------------------

for sample in range(NUM_SAMPLES):

 obj_x = np.random.uniform(-1,1)
 obj_y = np.random.uniform(-1,1)
```

```
 Pinc,Ptot,ground_truth = forward_em_simulation(obj_x,obj_y)

 O_img = rti_reconstruction(Pinc,Ptot)

 target = resize(ground_truth,(p,q))

 np.save(f"dataset/inputs/input_{sample}.npy",O_img)
 np.save(f"dataset/targets/target_{sample}.npy",target)

 if (sample+1)%25 == 0:
  print(f"{sample+1} samples generated")

print("Dataset generation finished")
print("Elapsed time:",time.time()-start)

import numpy as np
import matplotlib.pyplot as plt

# choose sample index
idx = 0

# load data
rti_img = np.load(f"dataset/inputs/input_{idx}.npy")
target_img = np.load(f"dataset/targets/target_{idx}.npy")

print("RTI shape:", rti_img.shape)
print("Target shape:", target_img.shape)

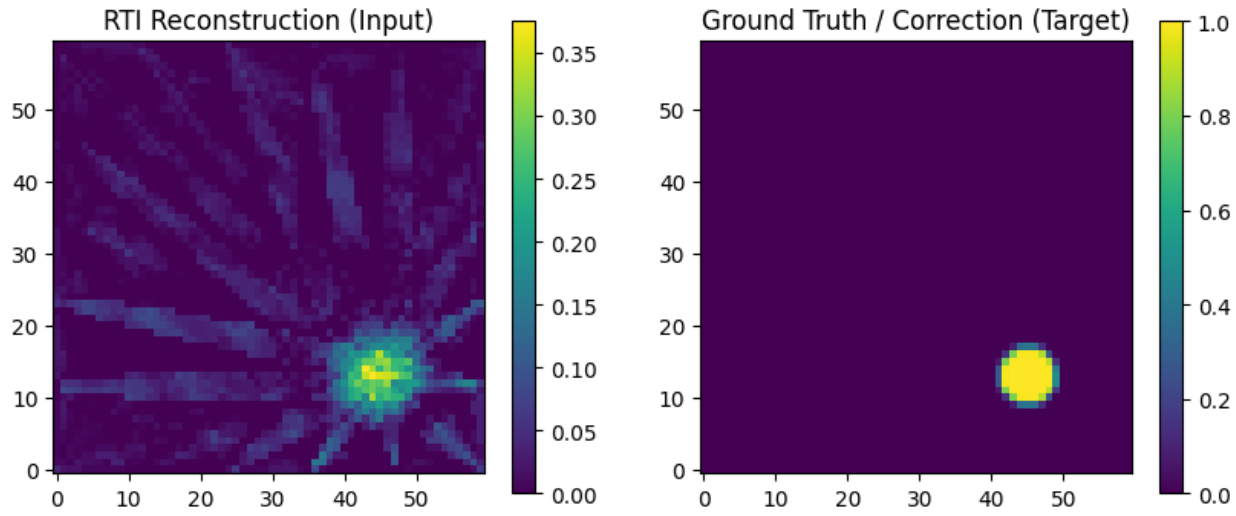
# plot
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(rti_img)
plt.colorbar()
plt.title("RTI Reconstruction (Input)")
plt.gca().invert_yaxis()

plt.subplot(1,2,2)
plt.imshow(target_img)
plt.colorbar()
plt.title("Ground Truth / Correction (Target)")
plt.gca().invert_yaxis()

plt.show()
```

```
RTI shape: (60, 60)
Target shape: (60, 60)
```



Training

```
from google.colab import files

uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving dataset.zip to dataset.zip
```

```
import zipfile
import os

zip_path = "dataset.zip"
extract_path = "dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted to:", extract_path)
```

```
Dataset extracted to: dataset
```

```
!ls dataset/dataset
```

```
inputs     targets
```

```
import torch
import torch.nn as nn

class DoubleConv(nn.Module):

    def __init__(self,in_ch,out_ch):
        super().__init__()
```

```
        self.net = nn.Sequential(
            nn.Conv2d(in_ch,out_ch,3,padding=1),
            nn.ReLU(),
            nn.Conv2d(out_ch,out_ch,3,padding=1),
            nn.ReLU()
        )

    def forward(self,x):
        return self.net(x)


class UNet(nn.Module):

    def __init__(self):
        super().__init__()

        self.down1 = DoubleConv(1,64)
        self.pool1 = nn.MaxPool2d(2)

        self.down2 = DoubleConv(64,128)
        self.pool2 = nn.MaxPool2d(2)

        self.middle = DoubleConv(128,256)

        self.up1 = nn.ConvTranspose2d(256,128,2,2)
        self.conv1 = DoubleConv(256,128)

        self.up2 = nn.ConvTranspose2d(128,64,2,2)
        self.conv2 = DoubleConv(128,64)

        self.out = nn.Conv2d(64,1,1)

    def forward(self,x):

        d1 = self.down1(x)
        p1 = self.pool1(d1)

        d2 = self.down2(p1)
        p2 = self.pool2(d2)

        m = self.middle(p2)

        u1 = self.up1(m)
        c1 = torch.cat([u1,d2],dim=1)
        c1 = self.conv1(c1)

        u2 = self.up2(c1)
        c2 = torch.cat([u2,d1],dim=1)
        c2 = self.conv2(c2)

        return self.out(c2)
```

```python
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
# from unet_model import UNet

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Optional: Print GPU info if available
if torch.cuda.is_available():
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory:
{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

class RTIDataset(Dataset):

    def __init__(self, n):
        self.n = n

    def __len__(self):
        return self.n

    def __getitem__(self, i):
        x = np.load(f"dataset/dataset/inputs/input_{i}.npy")
        y = np.load(f"dataset/dataset/targets/target_{i}.npy")

        x = torch.tensor(x).float().unsqueeze(0)
        y = torch.tensor(y).float().unsqueeze(0)

        return x, y

# Create dataset and dataloader
dataset = RTIDataset(1000)
loader = DataLoader(dataset, batch_size=8, shuffle=True)

# Move model to GPU
model = UNet().to(device)

# Optimizer and loss function
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
loss_fn = torch.nn.MSELoss()

# Training loop
for epoch in range(50):
    model.train()  # Set model to training mode
    total_loss = 0

    for batch_idx, (x, y) in enumerate(loader):
        # Move data to the same device as model
        x = x.to(device)
```

```
        y = y.to(device)

        # Forward pass
        pred = model(x)
        loss = loss_fn(pred, y)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        # Optional: Print progress every 100 batches
        if batch_idx % 100 == 0:
            print(f"Epoch {epoch}, Batch {batch_idx}, Loss:
{loss.item():.6f}")

    avg_loss = total_loss / len(loader)
    print(f"Epoch: {epoch}, Average Loss: {avg_loss:.6f}")

# Save model (save to CPU for compatibility when loading on CPU later)
torch.save(model.cpu().state_dict(), "unet_model.pth")
print("Model saved to unet_model.pth")

# Optional: Move model back to GPU if you want to continue training
# model = model.to(device)
```

```
Using device: cuda
GPU name: Tesla T4
GPU memory: 15.64 GB
Epoch 0, Batch 0, Loss: 0.015315
Epoch 0, Batch 100, Loss: 0.004121
Epoch: 0, Average Loss: 0.007082
Epoch 1, Batch 0, Loss: 0.003376
Epoch 1, Batch 100, Loss: 0.001125
Epoch: 1, Average Loss: 0.001626
Epoch 2, Batch 0, Loss: 0.000761
Epoch 2, Batch 100, Loss: 0.000294
Epoch: 2, Average Loss: 0.000479
Epoch 3, Batch 0, Loss: 0.000343
Epoch 3, Batch 100, Loss: 0.000247
Epoch: 3, Average Loss: 0.000239
Epoch 4, Batch 0, Loss: 0.000133
Epoch 4, Batch 100, Loss: 0.000110
Epoch: 4, Average Loss: 0.000163
Epoch 5, Batch 0, Loss: 0.000141
Epoch 5, Batch 100, Loss: 0.000226
Epoch: 5, Average Loss: 0.000135
Epoch 6, Batch 0, Loss: 0.000192
```

```
Epoch 6, Batch 100, Loss: 0.000122
Epoch: 6, Average Loss: 0.000138
Epoch 7, Batch 0, Loss: 0.000147
Epoch 7, Batch 100, Loss: 0.000105
Epoch: 7, Average Loss: 0.000121
Epoch 8, Batch 0, Loss: 0.000091
Epoch 8, Batch 100, Loss: 0.000162
Epoch: 8, Average Loss: 0.000116
Epoch 9, Batch 0, Loss: 0.000092
Epoch 9, Batch 100, Loss: 0.000059
Epoch: 9, Average Loss: 0.000101
Epoch 10, Batch 0, Loss: 0.000153
Epoch 10, Batch 100, Loss: 0.000086
Epoch: 10, Average Loss: 0.000100
Epoch 11, Batch 0, Loss: 0.000081
Epoch 11, Batch 100, Loss: 0.000077
Epoch: 11, Average Loss: 0.000103
Epoch 12, Batch 0, Loss: 0.000063
Epoch 12, Batch 100, Loss: 0.000108
Epoch: 12, Average Loss: 0.000127
Epoch 13, Batch 0, Loss: 0.000193
Epoch 13, Batch 100, Loss: 0.000077
Epoch: 13, Average Loss: 0.000087
Epoch 14, Batch 0, Loss: 0.000063
Epoch 14, Batch 100, Loss: 0.000058
Epoch: 14, Average Loss: 0.000083
Epoch 15, Batch 0, Loss: 0.000087
Epoch 15, Batch 100, Loss: 0.000110
Epoch: 15, Average Loss: 0.000085
Epoch 16, Batch 0, Loss: 0.000057
Epoch 16, Batch 100, Loss: 0.000093
Epoch: 16, Average Loss: 0.000080
Epoch 17, Batch 0, Loss: 0.000059
Epoch 17, Batch 100, Loss: 0.000072
Epoch: 17, Average Loss: 0.000076
Epoch 18, Batch 0, Loss: 0.000079
Epoch 18, Batch 100, Loss: 0.000104
Epoch: 18, Average Loss: 0.000095
Epoch 19, Batch 0, Loss: 0.000084
Epoch 19, Batch 100, Loss: 0.000079
Epoch: 19, Average Loss: 0.000174
Epoch 20, Batch 0, Loss: 0.000043
Epoch 20, Batch 100, Loss: 0.000063
Epoch: 20, Average Loss: 0.000080
Epoch 21, Batch 0, Loss: 0.000176
Epoch 21, Batch 100, Loss: 0.000065
Epoch: 21, Average Loss: 0.000070
Epoch 22, Batch 0, Loss: 0.000052
Epoch 22, Batch 100, Loss: 0.000074
```

```
Epoch: 22, Average Loss: 0.000068
Epoch 23, Batch 0, Loss: 0.000048
Epoch 23, Batch 100, Loss: 0.000052
Epoch: 23, Average Loss: 0.000070
Epoch 24, Batch 0, Loss: 0.000056
Epoch 24, Batch 100, Loss: 0.000054
Epoch: 24, Average Loss: 0.000069
Epoch 25, Batch 0, Loss: 0.000051
Epoch 25, Batch 100, Loss: 0.000113
Epoch: 25, Average Loss: 0.000073
Epoch 26, Batch 0, Loss: 0.000053
Epoch 26, Batch 100, Loss: 0.000061
Epoch: 26, Average Loss: 0.000070
Epoch 27, Batch 0, Loss: 0.000043
Epoch 27, Batch 100, Loss: 0.000078
Epoch: 27, Average Loss: 0.000065
Epoch 28, Batch 0, Loss: 0.000040
Epoch 28, Batch 100, Loss: 0.000132
Epoch: 28, Average Loss: 0.000068
Epoch 29, Batch 0, Loss: 0.000098
Epoch 29, Batch 100, Loss: 0.000079
Epoch: 29, Average Loss: 0.000061
Epoch 30, Batch 0, Loss: 0.000074
Epoch 30, Batch 100, Loss: 0.000054
Epoch: 30, Average Loss: 0.000067
Epoch 31, Batch 0, Loss: 0.000072
Epoch 31, Batch 100, Loss: 0.000063
Epoch: 31, Average Loss: 0.000064
Epoch 32, Batch 0, Loss: 0.000095
Epoch 32, Batch 100, Loss: 0.000047
Epoch: 32, Average Loss: 0.000068
Epoch 33, Batch 0, Loss: 0.000052
Epoch 33, Batch 100, Loss: 0.000058
Epoch: 33, Average Loss: 0.000071
Epoch 34, Batch 0, Loss: 0.000029
Epoch 34, Batch 100, Loss: 0.000069
Epoch: 34, Average Loss: 0.000066
Epoch 35, Batch 0, Loss: 0.000074
Epoch 35, Batch 100, Loss: 0.000071
Epoch: 35, Average Loss: 0.000174
Epoch 36, Batch 0, Loss: 0.000073
Epoch 36, Batch 100, Loss: 0.000052
Epoch: 36, Average Loss: 0.000059
Epoch 37, Batch 0, Loss: 0.000050
Epoch 37, Batch 100, Loss: 0.000062
Epoch: 37, Average Loss: 0.000057
Epoch 38, Batch 0, Loss: 0.000030
Epoch 38, Batch 100, Loss: 0.000079
Epoch: 38, Average Loss: 0.000055
```

```
Epoch 39, Batch 0, Loss: 0.000056
Epoch 39, Batch 100, Loss: 0.000051
Epoch: 39, Average Loss: 0.000057
Epoch 40, Batch 0, Loss: 0.000042
Epoch 40, Batch 100, Loss: 0.000062
Epoch: 40, Average Loss: 0.000056
Epoch 41, Batch 0, Loss: 0.000062
Epoch 41, Batch 100, Loss: 0.000081
Epoch: 41, Average Loss: 0.000073
Epoch 42, Batch 0, Loss: 0.000052
Epoch 42, Batch 100, Loss: 0.000073
Epoch: 42, Average Loss: 0.000054
Epoch 43, Batch 0, Loss: 0.000095
Epoch 43, Batch 100, Loss: 0.000055
Epoch: 43, Average Loss: 0.000055
Epoch 44, Batch 0, Loss: 0.000058
Epoch 44, Batch 100, Loss: 0.000044
Epoch: 44, Average Loss: 0.000054
Epoch 45, Batch 0, Loss: 0.000046
Epoch 45, Batch 100, Loss: 0.000034
Epoch: 45, Average Loss: 0.000053
Epoch 46, Batch 0, Loss: 0.000053
Epoch 46, Batch 100, Loss: 0.000050
Epoch: 46, Average Loss: 0.000055
Epoch 47, Batch 0, Loss: 0.000057
Epoch 47, Batch 100, Loss: 0.000067
Epoch: 47, Average Loss: 0.000051
Epoch 48, Batch 0, Loss: 0.000050
Epoch 48, Batch 100, Loss: 0.000058
Epoch: 48, Average Loss: 0.000058
Epoch 49, Batch 0, Loss: 0.000055
Epoch 49, Batch 100, Loss: 0.000045
Epoch: 49, Average Loss: 0.000061
Model saved to unet_model.pth
```

```
!ls
```

```
dataset  dataset.zip  sample_data  unet_model.pth
```

```
from google.colab import files
files.download('unet_model.pth')
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```

```
obj_x = np.random.uniform(-1,1)
obj_y = np.random.uniform(-1,1)

Pinc,Ptot,ground_truth = forward_em_simulation(obj_x,obj_y)
```

```
O_img = rti_reconstruction(Pinc,Ptot)

# target = resize(ground_truth,(p,q))

import torch
import numpy as np
import matplotlib.pyplot as plt
# from unet_model import UNet

# load trained model
model = UNet()
model.load_state_dict(torch.load("unet_model.pth"))
model.eval()

# run your forward simulation
# get RTI reconstruction

#O_img = ...   # regression output

x = torch.tensor(O_img).float().unsqueeze(0).unsqueeze(0)

with torch.no_grad():
    unet_out = model(x).squeeze().numpy()

# --------------------------------
# HYBRID COMBINATION
# --------------------------------

final = 0.5*O_img + 0.5*unet_out

plt.figure(figsize=(15,5))

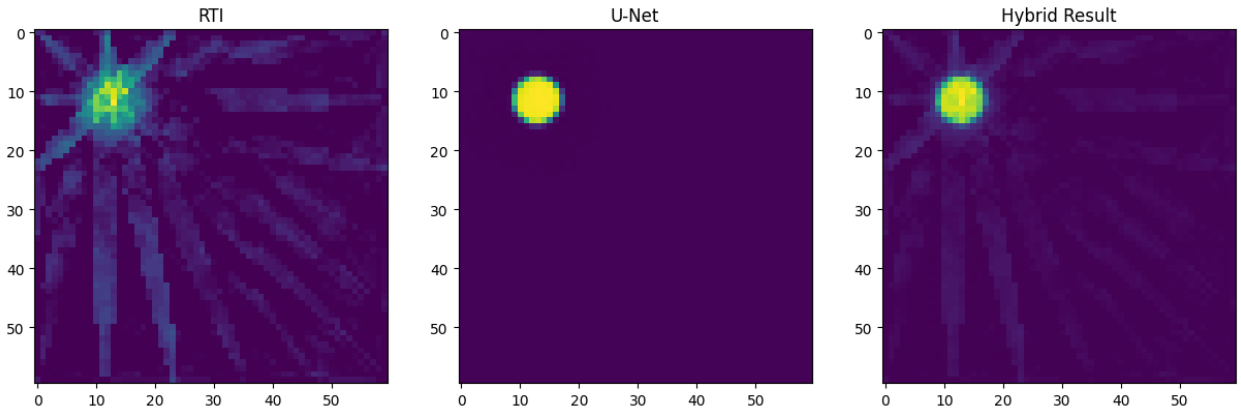
plt.subplot(1,3,1)
plt.title("RTI")
plt.imshow(O_img)

plt.subplot(1,3,2)
plt.title("U-Net")
plt.imshow(unet_out)

plt.subplot(1,3,3)
plt.title("Hybrid Result")
plt.imshow(final)



plt.show()
```




```
plt.plot(1,3,3)
plt.title("Ground Truth")
plt.imshow(ground_truth)
plt.show()
```

```
---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
/tmp/ipykernel_608/1845350093.py in <cell line: 0>()
----> 1 plt.plot(1,3,3)
      2 plt.title("Ground Truth")
      3 ground_truth = resize(ground_truth,(60,60))
      4 plt.imshow(ground_truth)
      5

NameError: name 'plt' is not defined
```